

TEORI BAHASA DAN AUTOMATA

Modul Pembelajaran: Konversi NFA ke DFA

Penjelasan step-by-step mengenai Subset Construction, cara kerja ϵ -Closure, serta panduan praktis merubah mesin NFA menjadi DFA yang setara.

Contents

1	Konsep Dasar Konversi Automata	2
1.1	Kenapa Harus Dikonversi?	2
1.2	Algoritma Subset Construction	2
2	Studi Kasus 1: NFA Biasa ke DFA	3
2.1	Proses Eksekusi Konversi	3
2.2	Hasil Akhir DFA	4
3	Bumbu Rahasia: ϵ-Closure	5
4	Studi Kasus 2: Epsilon-NFA ke DFA	6
4.1	Hasil DFA Epsilon-NFA	7

1 Konsep Dasar Konversi Automata

1.1 Kenapa Harus Dikonversi?

Di modul sebelumnya, kita udah bahas kalau NFA itu enak banget buat didesain oleh manusia karena sifatnya yang fleksibel, bisa bercabang, dan punya "jalan gaib" alias transisi epsilon (ϵ). Tapi sayangnya, komputer itu benci sama keraguan. Kalau kita mau bikin program betulan (kayak *compiler* atau alat pencari teks regex), komputer butuh instruksi yang serba pasti, satu per satu. Nah, instruksi yang pasti ini cuma dimiliki oleh **DFA**.

Untungnya, ada sebuah teori penting di ilmu komputer yang membuktikan bahwa: "**Setiap NFA pasti punya minimal satu DFA yang kemampuannya 100% setara**". Artinya, bahasa apa pun yang bisa dikenali sama NFA, pasti bisa juga dikenali sama DFA. Jadi, kita bisa menikmati enaknya ngedesain pakai NFA, lalu mengkonversinya ke DFA biar gampang dieksekusi sama komputer.

1.2 Algoritma Subset Construction

Cara atau algoritma buat ngubah NFA jadi DFA ini dikenal dengan nama **Subset Construction** (Konstruksi Subhimpunan).

Ide utamanya lumayan cerdas: Karena di NFA sebuah mesin bisa bercabang jadi beberapa kloningan dan ada di beberapa *state* sekaligus, maka kita bikin aja satu *state* DFA yang mewakili **gabungan (himpunan)** dari posisi kloningan-kloningan tersebut.

Logika Subset Construction

Bayangin di NFA, setelah baca karakter '0', kloningan mesin kamu nyebar dan lagi pada nangkring di *state* q_0 dan q_1 . Nah, di DFA, kita bikin satu *state* baru yang bentuknya emang himpunan, kita kasih nama aja *state* $\{q_0, q_1\}$. Biar nggak ribet nulisnya, himpunan $\{q_0, q_1\}$ ini bisa kita ganti namanya jadi *state* ****A**** atau *state* ****B****.

Aturan main merubah NFA jadi DFA secara general:

1. **Start State DFA** adalah himpunan yang isinya start state NFA (nantu ditambahin *epsilon-closure* kalau ada transisi epsilon).
2. **Transisi DFA** dievaluasi dari gabungan semua arah transisi masing-masing *state* NFA yang ada di dalam himpunan tersebut.
3. **Final State DFA** adalah *state* himpunan mana pun yang di dalamnya "mengandung" minimal satu final state dari NFA asalnya.

2 Studi Kasus 1: NFA Biasa ke DFA

Biar nggak bingung sama teorinya, mending langsung kita hajar pakai contoh. Kita bakal pakai NFA pendeteksi pola berakhiran "01" dari materi sebelumnya, alfabetnya $\Sigma = \{0, 1\}$.

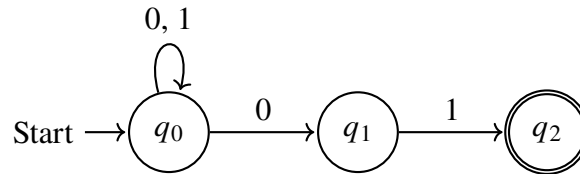


Figure 1: NFA Asal: Pola Akhiran "01"

Langkah pertama yang harus banget dilakukan adalah bikin **Tabel Transisi NFA**-nya dulu biar gampang dilihat:

State NFA	Input 0	Input 1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

2.1 Proses Eksekusi Konversi

Kita mulai bikin state DFA dari state awal NFA. Biar nggak bingung, state DFA kita kasih nama pakai huruf besar (A, B, C, dst).

Langkah 1: Tentukan Start State DFA

Start state DFA kita namakan state A. Isinya adalah start state NFA yaitu q_0 .

$$A = \{q_0\}$$

Langkah 2: Telusuri Transisi dari State A

Sekarang kita cek, kalau dari A dikasih input '0' dan '1', tujuannya ke mana aja berdasarkan tabel NFA di atas?

- A baca '0' $\Rightarrow$ lihat baris q_0 kolom 0 $\Rightarrow$ Hasilnya $\{q_0, q_1\}$. Ini kan himpunan baru nih, kita kasih nama state B.
- A baca '1' $\Rightarrow$ lihat baris q_0 kolom 1 $\Rightarrow$ Hasilnya $\{q_0\}$. Ini kan si state A itu sendiri.

Langkah 3: Telusuri State Baru (State B)

State B = $\{q_0, q_1\}$. Cara ngecek transisinya adalah dengan menggabungkan hasil transisi q_0 dan q_1 .

- B baca '0' $\Rightarrow$ transisi q_0 baca '0' **digabung** transisi q_1 baca '0'.
Hasilnya: $\{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$. Loh, balik lagi ke state B.
- B baca '1' $\Rightarrow$ transisi q_0 baca '1' **digabung** transisi q_1 baca '1'.
Hasilnya: $\{q_0\} \cup \{q_2\} = \{q_0, q_2\}$. Ini himpunan baru lagi! Kita kasih nama state C.

Langkah 4: Telusuri State Baru (State C)

State $C = \{q_0, q_2\}$.

- C baca '0' $\Rightarrow \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$. Balik ke state B .
- C baca '1' $\Rightarrow \{q_0\} \cup \emptyset = \{q_0\}$. Balik ke state A .

Karena udah nggak ada state himpunan baru lagi yang muncul, algoritmanya **selesai**.

Langkah 5: Tentukan Final State DFA

Lihat himpunan state A, B, dan C. Himpunan mana yang di dalamnya ada q_2 (final state asal NFA)?

- $A = \{q_0\}$ (Bukan final)
- $B = \{q_0, q_1\}$ (Bukan final)
- $C = \{q_0, q_2\}$ (**Ini FINAL STATE, karena mengandung q_2 !**)

2.2 Hasil Akhir DFA

Kalau kita rekap hasil corat-coret di atas, tabel transisi DFA-nya jadi rapi kayak gini:

State DFA	Isi Himpunan	Input 0	Input 1
$\rightarrow A$	$\{q_0\}$	B	A
B	$\{q_0, q_1\}$	B	C
$*C$	$\{q_0, q_2\}$	B	A

Dan diagram mesin DFA barunya bakal kelihatan seperti ini:

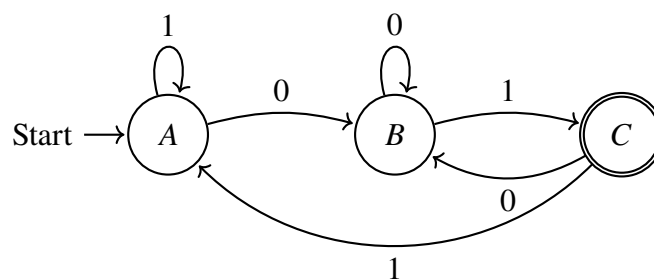


Figure 2: DFA Hasil Konversi (Setara dengan NFA Asal)

Gampang, kan? Cuma modal himpunan sama teliti aja baca tabel asalnya.

3 Bumbu Rahasia: ϵ -Closure

Nah, tadi kan konversi NFA yang polos-polos aja. Gimana kalau NFA-nya pakai transisi jalan tol alias ϵ -transition? Kita butuh jurus tambahan yang namanya **ϵ -Closure** (ditulis ϵ -Cl).

Apa itu ϵ -Closure?

ϵ -Closure dari sebuah state adalah **kumpulan semua state yang bisa didatengin dari state tersebut SECARA GRATIS cuma pakai transisi epsilon (ϵ)**. Jangan lupa, sebuah state itu selalu bisa mendatangi dirinya sendiri secara gratis. Jadi, nama state itu sendiri pasti masuk ke dalam himpunan ϵ -Closure-nya.

Contoh sederhana, kalau kita punya alur $q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} q_3$:

- $\epsilon\text{-Cl}(q_1) = \{q_1, q_2, q_3\}$ (Karena dari q_1 jalan ϵ tembus sampai q_3)
- $\epsilon\text{-Cl}(q_2) = \{q_2, q_3\}$
- $\epsilon\text{-Cl}(q_3) = \{q_3\}$ (Nggak ada panah ϵ keluar dari dia)

Setiap kali mesin mau gerak baca satu input karakter, dia harus ngecek ϵ -Closure **sebelum dan sesudah** pindah.

4 Studi Kasus 2: Epsilon-NFA ke DFA

Coba kita bedah contoh yang lumayan elegan ini. Kita mau bikin DFA dari NFA yang mengenali bahasa $a^*b^*c^*$ (huruf 'a', terus huruf 'b', terus huruf 'c', masing-masing jumlahnya bebas, boleh nol). Alfabetnya $\Sigma = \{a, b, c\}$.

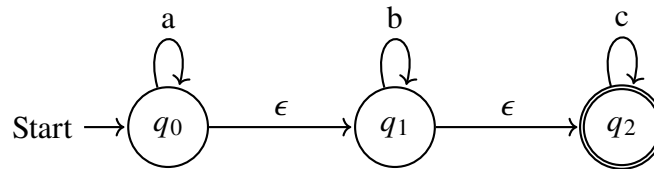


Figure 3: Epsilon-NFA untuk Pola $a^*b^*c^*$

Langkah 1: Hitung ϵ -Closure Tiap State

Biar enak, kita inventarisir dulu jalan gratis dari tiap state:

- $\epsilon\text{-Cl}(q_0) = \{q_0, q_1, q_2\}$ (Tembus dari awal sampai akhir!)
- $\epsilon\text{-Cl}(q_1) = \{q_1, q_2\}$
- $\epsilon\text{-Cl}(q_2) = \{q_2\}$

Langkah 2: Tentukan Start State DFA

Beda sama sebelumnya, start state DFA sekarang **bukan cuma** $\{q_0\}$, melainkan hasil $\epsilon\text{-Cl}(q_0)$. Begitu mesin dinyalain, dia langsung menjalar ke mana-mana. Kita namain state awal DFA ini A.

$$A = \epsilon\text{-Cl}(q_0) = \{q_0, q_1, q_2\}$$

Langkah 3: Cek Transisi dari State A

Rumus kalau mau ngecek transisi di Epsilon-NFA:

$$\delta_{\text{DFA}}(A, \text{input}) = \epsilon\text{-Cl}(\text{Gabungan pindahan tiap elemen di } A)$$

Evaluasi dari $A = \{q_0, q_1, q_2\}$:

- **Input 'a':**
Siapa di dalam $\{q_0, q_1, q_2\}$ yang punya panah 'a'? Cuma q_0 , tujuannya ke q_0 .
Kita tutup lagi pakai ϵ -Closure: $\epsilon\text{-Cl}(q_0) = \{q_0, q_1, q_2\}$.
Hasilnya balik lagi ke state A.
- **Input 'b':**
Siapa yang punya panah 'b'? Cuma q_1 , tujuannya ke q_1 .
Kita tutup pakai ϵ -Closure: $\epsilon\text{-Cl}(q_1) = \{q_1, q_2\}$.
Ini state baru, kita namain B.
- **Input 'c':**
Siapa yang punya panah 'c'? Cuma q_2 , tujuannya ke q_2 .
Kita tutup pakai ϵ -Closure: $\epsilon\text{-Cl}(q_2) = \{q_2\}$.
Ini state baru juga, kita namain C.

Langkah 4: Telusuri State B = $\{q_1, q_2\}$

- **Input 'a':** Nggak ada yang punya panah 'a'. Hasilnya Himpunan Kosong $\emptyset$. Kita sebut state ini D (Dead State).

- **Input 'b':** Cuma q_1 yang punya. Hasilnya $\epsilon\text{-Cl}(q_1) = \{q_1, q_2\}$. Balik ke state B .
- **Input 'c':** Cuma q_2 yang punya. Hasilnya $\epsilon\text{-Cl}(q_2) = \{q_2\}$. Maju ke state C .

Langkah 5: Telusuri State C = $\{q_2\}$

- **Input 'a':** Kosong $\Rightarrow D$
- **Input 'b':** Kosong $\Rightarrow D$
- **Input 'c':** Punya panah 'c'. $\epsilon\text{-Cl}(q_2) = \{q_2\}$. Balik ke C .

Langkah 6: Telusuri State D = $\emptyset$ Karena Dead State, dikasih input 'a', 'b', atau 'c' bakal tetep di D .

Langkah 7: Tentukan Final State DFA

Syaratnya, himpunan yang memuat q_2 adalah Final State.

- $A = \{q_0, q_1, \mathbf{q_2}\} \Rightarrow \mathbf{FINAL}$
- $B = \{q_1, \mathbf{q_2}\} \Rightarrow \mathbf{FINAL}$
- $C = \{\mathbf{q_2}\} \Rightarrow \mathbf{FINAL}$

Wow! State A, B, dan C semuanya jadi Final State.

4.1 Hasil DFA Epsilon-NFA

Hasil akhir diagram transisinya bakal jadi seperti ini:

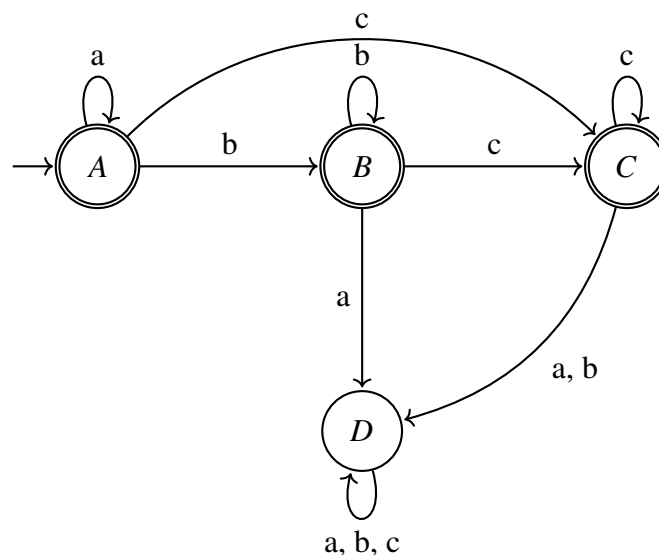


Figure 4: DFA Hasil Konversi Epsilon-NFA $a^*b^*c^*$

Dari sini kita bisa lihat betapa rapinya DFA menutupi kerumitan jalan gaib si ϵ . Mesin DFA di atas serba pasti, nggak ada jalan nyabang, dan bisa langsung di-coding ke dalam bahasa pemrograman betulan!